

Como o sistema de legendas do Clipper funciona

Documento-fonte da verdade — reavaliação completa sem nenhuma alteração de código. Data: 2026-09-19.
Regra da etapa: só entendimento; hipóteses marcadas como hipóteses.

1. Visão geral

O Clipper transforma uma live/gameplay em cortes verticais legendados. A legenda nasce do áudio, nunca do vídeo: o áudio é extraído, transcrito pelo Whisper (local, GPU Intel B580 via whisper.cpp/Vulkan), e cada palavra reconhecida vira um objeto `Word(text, start, end)` com timestamps em segundos absolutos do vídeo original. Todo o resto do sistema (scoring, seleção, agrupamento, SRT/ASS, FFmpeg) apenas recorta, agrupa e desloca esses timestamps — com uma exceção histórica importante (o VAD, removido do caminho padrão), nenhuma etapa "inventa" tempo.

A frase que resume o sistema:

> O Whisper mede o tempo; o pipeline só recorta e translada.

Quando a legenda aparece no momento errado mas com o texto certo, a pergunta correta nunca é "qual offset aplicar?", e sim "quem mediu esse tempo, e em que timeline?".

2. Fluxo completo

Fluxo real encontrado no código (nada aqui é presumido; cada etapa cita arquivo e função):

```
vídeo original
| core/backends.py: audio_chunks() - ffmpeg -ss <off> -t 30, 16 kHz mono
▼
chunks WAV de 30 s (+ offset em segundos)
| core/backends.py: _whisper_cmd() + transcribe_vulkan() - 1 processo
| whisper-cli por chunk, SEM flags de VAD
▼
JSON por chunk: transcription[{text, offsets{from,to}ms, tokens[{text, offsets{from,to}ms, p}]]]
| parse em transcribe_vulkan(): offsets ms→s + chunk_offset; filtra [eot]/vazios
▼
list[Segment(text, start, end, words)] - segundos ABSOLUTOS do vídeo
| core/transcribe.py: transcribe() - opcionalmente cache .cache/<fp>.json
▼
transcript global (concatenação; sem merge, sem dedup, sem retoque)
| core/candidates.py: build() - janelas [min_dur, max_dur], passo 20 s
| core/candidates.py: snap_all() - snap ±1,5 s + pad, clamp no MAX
▼
Candidate(start, end, text, words) + --pad
| core/audio.py: energia · core/scoring.py: nota/título (LLM, texto apenas)
| core/selection.py: select() - NMS + diversidade
▼
clipes escolhidos: clip_start / clip_end (absolutos)
| core/video.py: _group_cues() - words→cues (absoluto, mesma timeline)
| core/video.py: _split_lines() - ÚNICA conversão abs→rel (subtrai clip_start)
▼
SRT / ASS (tempos relativos ao corte, 0-based)
| core/video.py: cut() - trim/atrim + setpts + filtro subtitles, legenda queimada
▼
vídeo final .mp4
```

Observação de leitura: "absoluto" = segundos desde o início do vídeo original; "relativo" = segundos desde o início do corte (clip_start vira 0).

3. Whisper

3.1 O que o Whisper realmente produz

O `whisper.cpp` decodifica o áudio em duas estruturas independentes por chunk (verificado em `thirdparty/whisper.cpp/src/whisper.cpp` e `examples/cli/cli.cpp`):

- Segmento: bloco de texto com `t0/t1` vindos dos `*timestamp tokens*` do decodificador (as marcas `<|0.00|>`, `<|3.24|>` que o modelo emite entre trechos). É uma fronteira de `*decodificação*`, não de fala: o segmento termina onde o modelo decidiu cortar o texto, e pode cobrir silêncio antes/depois.
- Token/palavra: cada token carrega `t0/t1` próprios, estimados por alinhamento de atenção cruzada (cada token "olha" para quais frames de áudio ele corresponde). É uma medição `*por token*`, independente das fronteiras do segmento.

No JSON -o j f (o que o Clipper consome), cada item tem `offsets{from, to}` em milissegundos no nível do segmento e, dentro de `tokens[]`, `offsets` também em milissegundos por token, mais `p` (probabilidade do token).

3.2 O que significam start, end, degenerado, absoluto, relativo

- `Word.start/Word.end`: segundos (float) do vídeo original em que a palavra foi ouvida, segundo o alinhamento do token.
- Timestamp degenerado: `end <= start` (ex: `20.760 → 20.760`). Intervalo vazio: a palavra não ocupa tempo nenhum, então nenhuma duração de legenda pode ser derivada dela sem inventar tempo. Origem típica: tokens de continuação/hallucinação empilhados no último timestamp válido.
- Absoluto: segundos desde o 0 do vídeo original (ex: palavra ouvida aos `32.37` s do vídeo → `start=32.37`).
- Relativo: segundos desde o 0 do `*corte*` (mesma palavra num clip que começa em `30.0 → 2.37`). Conversão: `rel = abs - clip_start`. Só existe UMA conversão no sistema inteiro (`_split_lines()`, `core/video.py:283`).

3.3 Como o Clipper interpreta o JSON (core/backends.py:163-232)

Para cada token não-vazio e não-especial (`[eot]`, `[sot]` etc. descartados):

```
Word.start = chunk_offset + token.offsets.from / 1000
Word.end   = chunk_offset + token.offsets.to   / 1000
```

Para cada segmento: `Segment.start/end` recebe a mesma fórmula sobre os offsets do segmento, e `Segment.text` é o texto limpo. Se um segmento vier sem tokens válidos, as palavras são distribuídas uniformemente no intervalo (`_even_words` — único caso de tempo "fabricado", honesto e documentado no código).

Ponto crucial, verificado no código: as captions usam exclusivamente os timestamps das Words; os timestamps de Segmento são usados apenas para definir as janelas candidatas (`candidates.build()` usa `segments[k].start/end`). Segmento e palavra medem coisas diferentes (seção 12 mostra isso com dados).

4. VAD

4.1 O que é e por que existe

VAD (`*Voice Activity Detection*`) é um classificador que decide, a cada janela de áudio (~dezenas de ms), "tem voz humana aqui? sim/não". O modelo usado era o Silero (`models/ggml-silero-v5.1.2.bin`). Motivo de existir no `whisper.cpp`: não desperdiçar decodificação com silêncio/ruído e não alucinar texto em trecho sem voz — em tese, o Whisper recebe só os trechos com fala.

4.2 Onde entrava no pipeline (estado: REMOVIDO do caminho padrão)

Antes da remoção, `transcribe_vulkan()` acrescentava `-vm <modelo> --vad` ao comando. Hoje `core/backends.py:_whisper_cmd()` monta o comando sem nenhuma flag de VAD, e o fallback CPU usa `vad_filter=False` (`core/transcribe.py:49`). O arquivo do modelo continua no disco, mas nada o carrega. O laboratório (`core/translab.py`) nunca teve VAD em nenhum caminho.

4.3 O que cada parâmetro significava

Parâmetro	Default	Efeito real no whisper.cpp
--- --- ---		
<code>threshold (-vt)</code>	0.50	probabilidade mínima p/ frame contar como fala
<code>min_speech_duration_ms</code>	250	trechos de fala mais curtos são descartados
<code>min_silence_duration_ms (-vsd)</code>	100	silêncio mínimo p/ encerrar um trecho de fala
<code>max_speech_duration_s</code>	infinito	nunca dividia por duração
<code>speech_pad_ms (-vp)</code>	30	padding antes/depois de cada trecho
<code>samples_overlap</code>	0.10 s	overlap ao recortar amostras

4.4 Como o VAD afetava NOSSO pipeline (mecanismo verificado em código)

Este é o ponto mais importante do documento. Com VAD ativo, o `whisper.cpp`:

1. Recorta o áudio em trechos de fala e decodifica cada trecho separadamente — o Whisper "ouve" um áudio comprimido (silêncios removidos), com sua própria timeline interna. 2. Os timestamps de segmento no JSON são remapeados para a timeline original (`whisper_full_get_segment_t0/t1_from_state → map_processed_to_original_time`, `src/whisper.cpp:8100+`). 3. Os timestamps de token no JSON NÃO são remapeados: `output_json` usa `whisper_full_get_token_data()`, que retorna `result_all[...].tokens[...]` crus (`src/whisper.cpp:8221`), ou seja, tempos na timeline comprimida.

Consequência, verificada nas duas pontas do código: com VAD, segmentos e palavras do mesmo JSON viviam em timelines diferentes. E o Clipper usa segmentos para as janelas (`candidates.build`) e palavras para as captions (`_group_cues`). Texto certo no momento errado era o resultado esperado dessa arquitetura — não um mistério, e não corrigível com offset (o deslocamento varia por trecho, conforme o quanto de silêncio foi removido antes dele).

Evidência medida (`docs/vad-experiment.md`, reproduzível): onset adiantado em ~0.67 s, cauda de fala real colapsada num ponto (som 26.98–29.95 virou 25.49–25.49), e em gameplay ~24 s de fala apagados com 75% de tokens degenerados. Por isso o VAD saiu do caminho padrão — decisão por evidência, não por preferência.

5. Chunks

5.1 Por que existem

O Whisper foi projetado para janelas de 30 s (`CHUNK_SECONDS = 30`, `core/backends.py:19`). `audio_chunks()` extrai via `ffmpeg` (`-ss <off> -t 30`, 16 kHz mono, ~1 MB por chunk, removidos após uso) sem carregar o vídeo na RAM, e cada chunk roda 1 processo `whisper-cli` sequencial. Sem chunks, vídeos de horas não caberiam no fluxo; com chunks, cada inferência é barata (~3 s/30 s de áudio na B580).

5.2 `chunk_offset`, timestamps locais e conversão

O `whisper-cli` sempre acha que está ouvindo "do segundo 0" — os offsets do JSON são locais ao chunk. A conversão para absoluto é uma soma, feita só no parse (`core/backends.py`, seção 3.3):

$\text{absoluto} = \text{chunk_offset} + \text{local_ms} / 1000$

onde `chunk_offset` é o -ss usado na extração (0, 30, 60, ...).

5.3 Como um erro de offset poderia acontecer (e por que não é o culpado)

Um erro aqui somaria ± 30 s (um chunk inteiro) a todas as palavras de um chunk — erro grosseiro e discreto, impossível de confundir com "alguns décimos de desalinhamento". Testes: transcrição full vs chunked difere 0.01–0.02 s, e o laboratório mediu chunks vs global idênticos (107/107 words, $\Delta=0.000$). Classificação: DESCARTADO como causa do problema atual.

6. Timestamps — tabela de todas as transformações

#	Etapa (arquivo:função)	Entrada	Transformação	Saída	Pode alterar?
--- --- --- --- ---					
1	whisper.cpp (decodificador)	áudio do chunk	alinhamento atenção → token t0/t1 (centissegundos)	JSON offsets ms (locais)	SIM — é a medição original (jitter natural ~0.5 s)
2	backends.py: transcribe_vulkan (parse)	offsets ms + chunk_offset	off/1000 + offset; filtra especiais/vazios	Word/Seconds absolutos	NÃO (soma exata; filtro só remove)
3	backends.py: _even_words	segmento sem tokens	divisão uniforme do intervalo	Words distribuídas	SIM — tempo fabricado (caso raro, honesto)
4	transcribe.py + cache.py	list[Segment]	serializa/reconstrói float	mesmo transcript	NÃO (copia; fingerprint path+size+mtime+modelo+idioma)
5	candidates.py: build	Segment.start/end	janelas ≤ max_dur, passo 20 s	Candidate(start,end) abs	NÃO cria tempo de palavra; define a janela do clip
6	candidates.py: _snap_one	Candidate + words	snap ±1,5 s à palavra + pad; clamp [min,max] e [0,media_end]	Candidate ajustado	SIM — move as BORDAS do clip (nunca o tempo das words)
7	video.py: _group_cues	words + clip bounds	filtra fora do clip e degenerados; agrupa por gap/pontuação; piso visual 1 s com clamp	cues (s,e,texto) absolutas	Duração da cue é derivada (piso 1 s), mas o INÍCIO é sempre o start de uma word real
8	video.py: _split_lines	cues absolutas	rel = abs - clip_start; clamp [0,duration]; quebra linhas	cues relativas	NÃO (subtração exata; clamp só corta excesso)
9	video.py: build_srt/build_ass	cues relativas	formatação de tempo + estilos	texto SRT/ASS	NÃO (só formata)
10	video.py: cut (ffmpeg)	vídeo + SRT/ASS + clip bounds	trim/atrim + setpts (0-based) + filtro subtitles	.mp4 final	NÃO (timeline 0-based alinhada; medido ponta a ponta)

Leitura da tabela: depois da medição (#1, e o raro #3), nenhuma etapa move o instante de uma palavra. Bordas de clip (#6) e durações de cue (#7) são derivadas, mas sempre ancoradas em timestamps reais.

7. Transcript

7.1 Transcript global (criação exata)

`transcribe()` (`core/transcribe.py:61`) retorna a concatenação, em ordem, dos `Segment` de todos os chunks. Não há merge (dois segmentos adjacentes continuam dois), não há deduplicação, não há retoque de timestamps, não há reordenação além da ordem dos chunks. "Global" aqui significa apenas "a lista completa do vídeo inteiro".

7.2 Cache (e se ele contamina execuções)

Com `--cache-dir`, o transcript é salvo em `.cache/<sha1>.transcript.json` e reutilizado se o fingerprint coincidir (caminho + tamanho + mtime + modelo + idioma). Efeito: trocar o arquivo invalida sozinho; mas editar o modelo ou o código de parse sem trocar o vídeo reaproveita transcript velho — ao investigar timestamps, use `--force-retranscribe` ou desconfie do cache primeiro. O cache guarda floats JSON (precisão total); não arredonda.

7.3 Representações

- `Word(text, start, end)` (`core/models.py`) — unidade: segundos float.

`text` preserva o espaço à esquerda do token ("`direita`" = fronteira real de palavra; sem ele, `smart_join` não remontaria "direita").

- `Segment(text, start, end, words)` — bounds do decodificador + lista de Words.
- `transcript.json` do cache / `work/<video>/transcription/` na revisão humana

(status `draft/approved`; a aprovada é a fonte da verdade do pipeline de revisão — `core/review.py`).

8. Seleção dos clips

1. `candidates.build()` (seção 6, #5): janelas deslizantes sobre os segmentos, duração dentro de `[--min-duration, --max-duration]` (default 20–90 s). 2. `snap_all()`: cada borda "gruda" na fronteira de palavra mais próxima (± 1.5 s) e ganha `--pad` (default 0.8 s) de respiro; o resultado é clampado no MAX e expandido (dentro da mídia) até o MIN. `--pad` nunca estoura o MAX. 3. `annotate` (`energia`) + `scoring.py`: o LLM recebe só texto (trecho + duração/energia/speech_rate) e devolve nota/título/hashtags. O `scoring` não toca em timestamps; o prompt exige título-recorte fiel ao texto. 4. `selection.select()`: filtra por nota, ordena, aplica NMS com decaimento por overlap + diversidade (máx N por 10 min). Não move bordas.

`clip_start/clip_end` finais = bordas snapped (seção 6, #6). `--pad` entra apenas aí, como respiro simétrico antes/depois.

9. Geração das captions

9.1 Word → Cue → SRT/ASS (conceitos)

- Word: uma palavra ouvida, com instante medido (seção 3.2).
- Cue: um cartaz de legenda — intervalo + texto ("o que aparece na tela, de quando a quando"). Uma cue agrupa várias words.
- SRT/ASS: o mesmo conjunto de cues serializado em dois formatos

(SRT simples; ASS estilizado com lanes/animação/hook). Mesmo núcleo, mesma timeline — SRT == ASS é invariante testada.

9.2 O que `_group_cues()` faz, exatamente (core/video.py:218)

1. Ordena as words por start; descarta degeneradas (`end <= start`) e as fora de `[clip_start, clip_end]` — ponto único de filtragem. 2. Agrupa em sequência: quebra o grupo quando (a) `gap > 0.4 s` (`CAPTION_PAUSE_THRESHOLD`), (b) palavra termina com `. ! ?` (e grupo tem ≥ 2), (c) grupo atinge 8 words, ou (d) texto passa de 48 chars. 3. Cada grupo vira cue: início = start da primeira word (sempre real); fim = end da última word, estendido a 1.0 s de piso visual se menor (`CAPTION_MIN_DURATION`). 4. Clamp: a parte *artificial* da extensão nunca invade a próxima cue (`commit d3f844d`); span real nunca é reduzido; fim == início é permitido.

9.3 O que `_split_lines()` faz, exatamente (core/video.py:283)

1. Converte: `s = abs - clip_start`, clamp em `[0, duration]` — a ÚNICA conversão `abs`→`rel` do sistema. 2. Reaplica piso de 1 s + clamp contra a próxima cue (versão relativa da mesma regra do item anterior — o clamp absoluto poderia ser re-estendido aqui). 3. Quebra o texto em linhas (≤ 24 chars, ≤ 2 por bloco) e, se houver mais de um bloco, divide a duração igualmente entre eles.

10. SRT e ASS

`build_srt()` (core/video.py:317) numera as cues e formata `HH:MM:SS,mmm --> HH:MM:SS,mmm`. `build_ass()` (core/video.py:503) emite o mesmo conjunto com estilos (fonte Montserrat ExtraBold, PlayRes = frame real, pop de 280 ms, hook com o título nos primeiros 2.8 s, lanes anti-overlap por MarginV, destaque amarelo de palavras do título). Eventos acústicos (*ÁUDIO ESTOURADO*, `opt-in`) fundem-se via `merge_event_cues()` (core/video.py:255): palavra sempre vence, evento é cortado onde cobre fala. Nenhum dos dois formatos altera tempos — só serializam as cues já relativas.

11. FFmpeg

`cut()` (core/video.py:801) corta com `trim=start=fim + setpts=PTS-STARTPTS` (áudio: `atrim + asetpts`) — corte 0-based frame-accurate — e queima a legenda com o filtro `subtitles`. A medição ponta a ponta que justifica o método está comentada no código: `-ss` de saída deslocaria a timeline das legendas (o filtro `subtitles` avalia na timeline do demux), enquanto `trim+setpts` deixa vídeo, áudio e legendas todos 0-based e alinhados. Encode `h264_qsv` (B580) com fallback `libx264`; composição vertical 1080x1920 com faixas blur do próprio vídeo. O FFmpeg não move legenda: ele apenas alinha o zero do corte com o zero do arquivo de legenda.

12. Caso real 55s → 30s (estudo de caso, sem correção)

Dados: `work/videofull/transcription/transcript.raw.json` (gerado na era VAD), segmento 1:

```
{ "text": "Agora eu vou matar esse cara que vai se dar picaretada. Quero nem saber.",
  "start": 30.0, "end": 58.61 }
```

Respostas às 10 perguntas, todas verificadas no dado + código:

1. Sim — start/end são timestamps de SEGMENTO (fronteiras do decodificador, `whisper_full_get_segment_t0/t1`; seção 4.4: com VAD, valores remapeados para a timeline original).
2. Sim — há 19 tokens dentro, com offsets próprios. 3. As words vão de 30.00 ("Agora" 30.00→32.35) até 47.80; as 7 últimas ("ada", ".", "Qu", "ero", "nem", "saber", ".") estão todas colapsadas em 47.80→47.80 — continuação hallucinada empilhada no último timestamp válido (mecanismo documentado em `docs/caption-timing.md` §6).
4. Sim — começam em 30.0, não em 55. 5. Não aplicável (ver item 4).
6. O 58.61 pertence ao SEGMENTO, a nenhuma palavra (última word termina em 47.8 — quase 11 s de "texto sem tempo"). É o fim da janela de decodificação, possivelmente cobrindo silêncio pós-fala. 7. Produzido pelo `whisper.cpp` com VAD, chunk 30–60 s de `videofull.mkv` (segmentos 0/1/2 = chunks 0–30/30–60/60–88; offsets locais + 30).
8. Decodificador do `whisper.cpp` (timestamp tokens → `result_all[i].t0/t1`).
9. O Clipper interpreta assim: `Segment(30.0, 58.61)` define a janela

candidata (`candidates.build` usa bounds de segmento); as captions usam só as words (30.0 → 47.8, com as 7 degeneradas descartadas por 21163e9). 10. Conversão: `Word = chunk_offset(30) + token_ms/1000`, idêntica à seção 3.3.

Leitura do caso: o objeto diz "o decodificador emitiu esse texto na janela 30 → 58.6, mas só mediu tempo de palavra até 47.8". O trecho "Quero nem saber" existe como `*texto*` (tokens), mas não existe como `*tempo*` (todos degenerados) — por isso foi descartado das captions corretamente, e por isso "a legenda não aparece" nesse trecho mesmo com o texto certo no transcript. Quanto ao "55s ouvido": o dado posiciona a fala em 30–47.8 do vídeo; se a audição indica ~55 s, a divergência está na medição original do Whisper (§14, item "desconhecido") — o pipeline posterior só propagou esses números.

Corolários do mesmo arquivo: segmento 0 tem UMA word (" é" 0.0→17.24, 17 s para uma palavra — medição grosseira do modelo, não erro de pipeline) com bounds de segmento (2.43 → 29.98) diferentes dos da word — mais uma prova de que segmento ≠ palavra. A transcrição aprovada manteve a estrutura (revisão humana não corrigiu tempos), e os títulos ("Agora eu vou matar esse cara") estão sustentados pelo texto aprovado (grounding OK).

13. O que já foi corrigido (e não deve ser revertido)

Commit	Problema real	Correção
--- --- ---		
21163e9	tokens end<=start viravam cues artificiais de 1 s em silêncio real	descarta degenerados em <code>_group_cues()</code> (ponto único)
d3f844d	extensão visual de 1 s invadia a próxima cue (24 overlaps medidos)	clamp só da parte artificial em <code>_group_cues() + _split_lines()</code>
sem-VAD padrão	VAD apagava fala e colapsava timestamps (§4.4, §12)	<code>_whisper_cmd()</code> sem flags VAD; <code>vad_filter=False</code> no fallback CPU
range de clips	clips de até ~90 s sem controle	<code>-min/max-duration (20/90)</code> , MAX teto real (<code>build+snap+pad</code>)
revisão humana	título podia inventar fatos sobre erro de reconhecimento	transcrição aprovada = fonte da verdade; grounding validado; prompt exige título-recorte

Descartados com evidência e fora do sistema: offset global, DTW (pior que offsets no onset, custo 1.2–2.2×), média de timestamps, fusão heurística.

14. O que os experimentos realmente provaram

Conclusão anterior	Veredito	Base
--- --- ---		
timestamps degenerados viram cues falsas	CONFIRMADO	código + overlaps 10 → 0 (GuilhermeGaivota)
extensão de 1 s invadia próxima cue	CONFIRMADO	24 → 0 overlaps (VideoMedio2)
conversão abs → rel correta	CONFIRMADO	CUE-WORD = +0.000 em todos os testes
chunk offset correto	CONFIRMADO	full vs chunks Δ 0.01–0.02 s; lab 107/107 Δ=0
FFmpeg/PTS correto	CONFIRMADO	start_time 0.0; trim+setpts medido
padding correto	CONFIRMADO	caption em 0.8 s com pad 0.8 é correto
SRT == ASS	CONFIRMADO	mesmo núcleo (invariante testada)

timestamps degenerados viram cues falsas	CONFIRMADO	código + overlaps 10 → 0 (GuilhermeGaivota)
VAD prejudica timestamps	CONFIRMADO	onset -0.67, cauda colapsada, fala apagada (reproduzido 2×) + mecanismo em código (§4.4)
threshold (-vt) causa o corte	DESCARTADO	onset idêntico em 0.2/0.5/0.8
-vp alto resolve sem risco	NÃO CONFIRMADO	VP300 recupera cauda mas VP500 gera timestamps fora do chunk (inseguro)
DTW como solução	DESCARTADO	t_dtw pior no onset; não recupera cauda
chunking degrada	DESCARTADO	chunks == global
contexto ajuda	DESCARTADO	context 5 diverge Δ~0.56 s
filtros de áudio melhoram	NÃO CONFIRMADO	0 deg em todos; diferenças laterais; agressivos derrubam confiança
clipping detectável	CONFIRMADO	limpo=0 eventos; gameplay com conf 0.5–1.0
risada detectável por volume	DESCARTADO (como detector)	sem assinatura confiável; não implementada
pausas preservadas	CONFIRMADO	curta/média/longa mantidas em todas as configs

15. O que ainda é desconhecido

1. Por que o Whisper posicionou a fala do caso §12 em 30–47.8 (se a audição de ~55 s estiver correta): erro de medição do modelo, ou referência equivocada de quem ouviu. Teste necessário: ouvir o trecho 25–60 s do `videofull.mkv` marcando início real da frase e comparar com 30.0. 2. Words de span absurdo ("é" 0→17.2): quanto o alinhamento por token erra em vogais isoladas/interjeições, e se há sinal (ex: p baixo) que permita sinalizar sem inventar tempo. Precisa de medição, não de palpite. 3. Jitter residual sem VAD (~0.5 s): origem interna do modelo; sem correção conhecida que não seja offset (proibido por evidência). 4. large-v3 vs medium: infraestrutura aceita (--whisper-model), mas nunca medido nesta máquina (modelo não baixado — e não baixar nesta etapa).

16. Próxima investigação recomendada

Primeira coisa, em ordem, cada uma pequena e com veredito antes da próxima:

1. Audição marcada do caso §12 (25–60 s de `videofull.mkv`): estabelecer a verdade acústica do início da frase. Se for ~30 s, o sistema está consistente e o "55 s" era referência errada — encerra o mistério sem código. Se for ~55 s, o erro é de medição do Whisper (origem), e o passo 2 decide o resto. 2. Retranscrever o chunk 30–60 sem VAD (pipeline atual) e comparar words: o default mudou depois desse `transcript.raw.json`; o dado do §12 é da era VAD e pode já estar superado. 3. Só então, e só com divergência reproduzida no pipeline atual: instrumentar (o `caption-debug.txt` já distingue `end <= start` de `outside_selected_clip`) e procurar a primeira divergência — nunca mexer em `_group_cues()` antes.

Código que NÃO deve ser alterado agora: `_group_cues()`, `_split_lines()`, `build_srt/build_ass`, `cut()/FFmpeg`, `_whisper_cmd()`, `candidates.build()` e os commits da seção 13. Experimentos válidos: todos os CONFIRMADO/DESCARTADO acima. Tratar com cautela: qualquer número vindo de transcript da era VAD (incluindo `work/videofull/`) e a ideia de que "mais words = melhor" nos presets agressivos.

Apêndice A — Mapa de responsabilidades

Componente	Responsabilidade	NÃO é responsabilidade dele
------------	------------------	-----------------------------

|---|---|---|

Whisper	medir QUANDO cada token foi ouvido; transcrever o quê	ser pontual além do jitter (~0.5 s); cobrir silêncio com tempo real
VAD (removido)	classificar voz/não-voz	preservar timelines (não preservava: §4.4)
chunking	viabilizar inferência em vídeos longos	medir tempo (só soma offset exato)
transcript/cache	guardar e devolver os números intactos	corrigir, mesclar ou reinterpretar
scoring/seleção	escolher ONDE cortar (texto + energia)	mover tempos de palavra
grouping (_group_cues)	agrupar words em cartazes; descartar o inválido	inventar instante (início é sempre word real)
SRT/ASS	serializar as mesmas cues em dois formatos	alterar qualquer tempo
FFmpeg	alinhar o zero do corte com o zero da legenda; queimar	deslocar timeline
revisão humana	corrigir TEXTO; aprovar a verdade	corrigir tempo do modelo
acoustic.py	sinalizar clipping com confiança	dizer o que foi falado no trecho estourado

Apêndice B — Por que o erro "parece progressivo" (e não é drift)

Se cada palavra tem timestamp próprio e correto, erro progressivo é impossível: A 10.0→10.3, B 10.4→10.8, C 10.9→11.2 — cada legenda ancora no seu instante, erro não acumula. O que PARECE progressivo (+0.0, +0.1, +0.2, +0.4, ... de docs/caption-timing.md) é a assinatura de timestamps errados na origem com erro variável por trecho (ex: mapeamento VAD comprimindo mais uns trechos que outros), não de alteração posterior — e a tabela da seção 6 mostra que nada depois da medição move instantes de palavra. Distinção final:

- Errados na origem: cada word carrega seu próprio erro (caso §12, jitter, era VAD). Sintoma: erros diferentes por trecho, sem padrão de fase.
- Alterados pelo pipeline: exigiria reescrita de `start` — o código não tem nenhum caminho que faça isso (verificado etapa por etapa na seção 6).